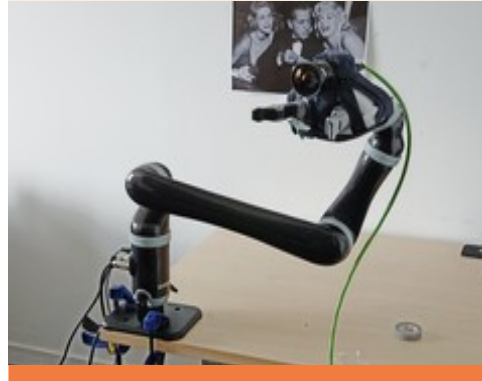


Navigation, Path & Motion Planning (Part 1)

VA56 - Introduction to Mobile robotics



UTBM - FISE INFO - VA56

Jocelyn BUISSON

Introduction & Definition

Motivation

- Mobile robots need to compute collision-free, feasible, and efficient motions autonomously
 - Robots operating in the real world must decide how to move from an initial location to a goal
 - While avoiding collisions
 - Respecting physical constraints
 - Optimizing some objective
 - Length, time, smoothness, energy, etc.*

Introduction & Definition

Motivation

- Challenges
 - Geometric complexity
 - Environments are cluttered

Narrow corridors, furniture, humans, vegetation, etc.
 - Obstacles may be irregular or dynamic

Collision checking happens in a continuous space, even if the map is discrete
 - Requires reasoning in configuration space

The robot has a volume and an orientation, we cannot treat it as a single point

Introduction & Definition

Motivation

- Challenges
 - Dimensionality
 - In 2D we have 3 degrees of freedom (DOF)
 - In 3D we have 6 DOF
 - Planning complexity grows exponentially
 - Orientation affects feasible directions which make planning a non-linear problem

Introduction & Definition

Motivation

- Challenges
 - Constraints
 - Collision constraints

The robot must maintain clearance from obstacles (imperfect localization => safety margins)
 - Kinematic constraints

Differential (cannot strafe sideways), Ackermann (minimum turning radius), non-holonomic
 - Dynamic constraints

Maximum velocity, maximum acceleration, tire friction limits, aerodynamics, etc.
 - Uncertainty in sensing and actuation

Localization noise, actuation errors (wheel slip, drift), incomplete maps, etc.

Introduction & Definition

Definitions

- Configuration space (C-space)

The configuration space $\mathcal{C}$ is the set of all possible configurations q of the robot.

- C-space obstacle and free space

- An obstacle in workspace maps to a region in configuration space:

$$\mathcal{C}_{obs} = \{q \in \mathcal{C} | A(q) \cap \text{obstacle} \neq \emptyset\}$$

Where $A(q)$ is the robot's footprint in configuration

- Free space is the configuration space without the obstacles

$$\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{C}_{obs}$$

- A valid motion must remain entirely inside $\mathcal{C}_{free}$

Introduction & Definition

Definitions

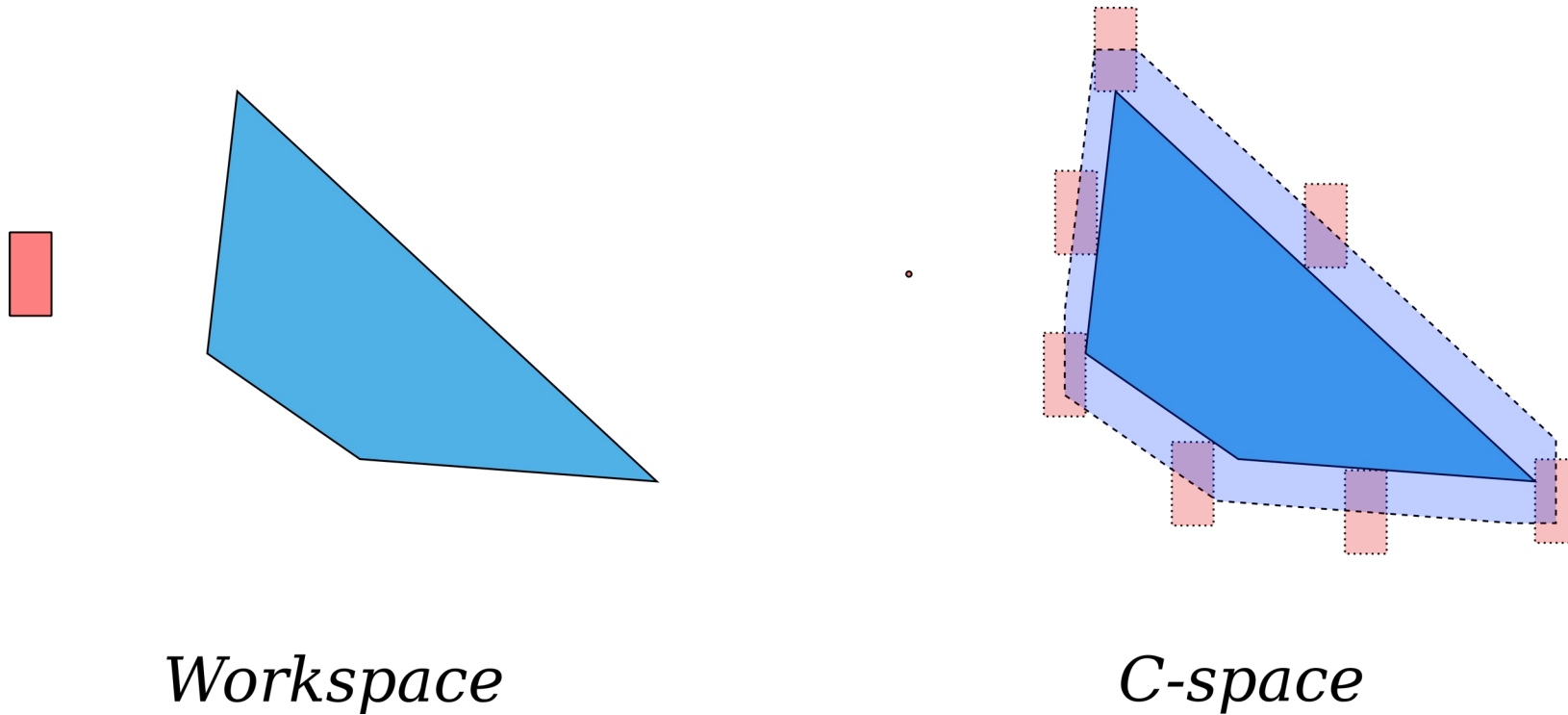


Figure 1: 2-DOF Workspace vs C-space

Introduction & Definition

Definitions

- Path vs Trajectory

- A path is a continuous curve in configuration space:

$$\sigma : [0, 1] \rightarrow \mathcal{C}_{free}$$

A path captures where the robot goes but not how fast

- A trajectory is time-parameterized path:

$$\tau : [0, T] \rightarrow \mathcal{C}_{free}$$

A trajectory must satisfy dynamic constraints (velocity, acceleration, control limits)

- Path planning vs Motion planning

- Given a map, start and goal configurations, path planning produces a geometry path in $\mathcal{C}_{free}$

Ignores time & dynamics

- Motion planning produces a feasible trajectory

Often implemented as: global geometric planning + local trajectory generation / control

Introduction & Definition

Problem formulation

Given:

- Configuration space $\mathcal{C}$
- Free space $\mathcal{C}_{free}$
- Initial configuration q_{start}
- Goal region $\mathcal{C}_{goal} \subset \mathcal{C}_{free}$

Find:

$$\sigma : [0, 1] \rightarrow \mathcal{C}_{free} \quad \text{such that} \quad \sigma(0) = q_{start}, \sigma(1) \in \mathcal{C}_{goal}$$

Find a continuous sequence of valid configurations leading to the goal

If dynamics are included, the problem becomes:

$$\dot{x} = f(x, u), \quad u \in \mathcal{U}$$

Find a continuous control sequence that generates a feasible trajectory to the goal

Introduction & Definition

Properties of planning algorithm

- Completeness

A complete algorithm is guaranteed to terminate and either (a) returns a solution if one exists, or (b) report correctly that no solution exists.

- Resolution-complete

Guarantees success if the resolution is sufficient

- Probabilistically-complete

Probability of success $\rightarrow 1$ as time $\rightarrow \infty$

Introduction & Definition

Properties of planning algorithm

- Optimality

- Optimal

- Always produces the globally optimal solution*

- Resolution-optimal

- Guarantees optimality as resolution $\rightarrow \infty$*

- Asymptotically-optimal

- Converges to the optimal solution as number of samples $\rightarrow \infty$*

Map & Environment Representation

Why multiple map representations

Planners require different properties for the environment representation:

- Efficient collision checking
- Efficient neighbor queries
- Ability to compute distances and heuristics
- Ability to reason about geometry (visibility, connectivity)
- Suitability for high-dimensional spaces
- Storage and runtime trade-offs

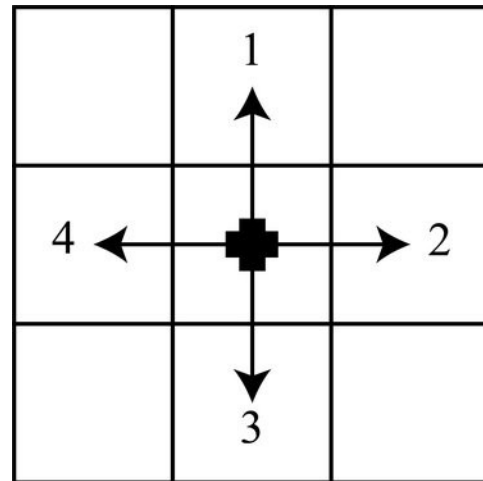
There is not single best map representation, the choice depends on the planning algorithm.

Map & Environment Representation

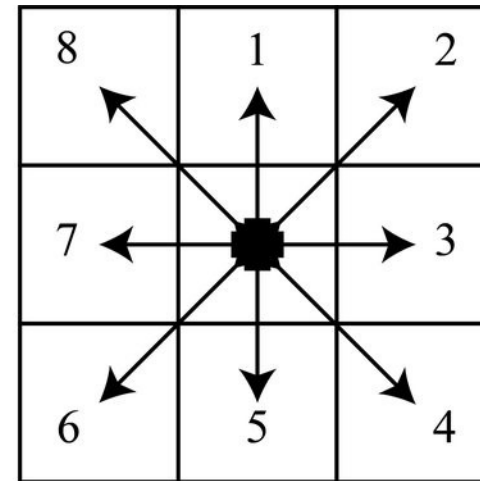
Grid maps & costmaps

Regularly discretizes space into cells

- Mainly two types
 - Grid map (occupancy grid): each cell has a discrete state (occupied / free / unknown)
 - Costmap: each cell has a cost related to proximity to obstacles
- Connectivity: cells are typically 4-connected or 8-connected



4-Connected



8-Connected

Map & Environment Representation

Grid maps & costmaps

- Usually 2D only (no orientation information), 3D grids are costly in memory & computation
- Robot footprint: grids can be inflated to account for robot size (often circular approximation)
- Grid resolution impacts path accuracy and computation cost
- Supports dynamic obstacles
- Very common in ROS

Map & Environment Representation

Grid maps & costmaps

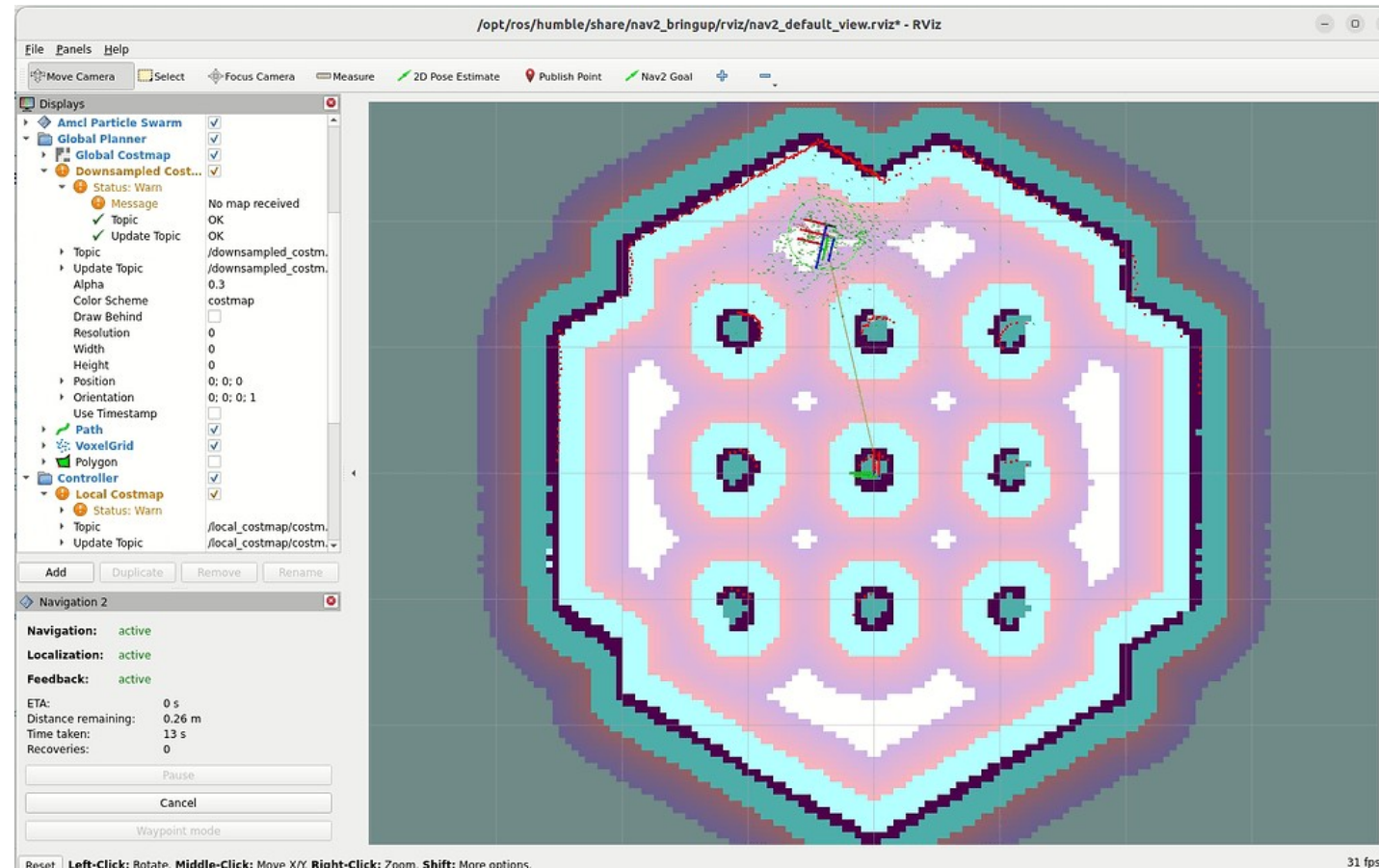


Figure 2: Costmap.

Map & Environment Representation

Topology-based representations (Roadmaps)

Abstract the environment as graph capturing connectivity but not full geometry.

- Voronoi diagram / Medial axis (edges equidistance from obstacles, maximum clearance)

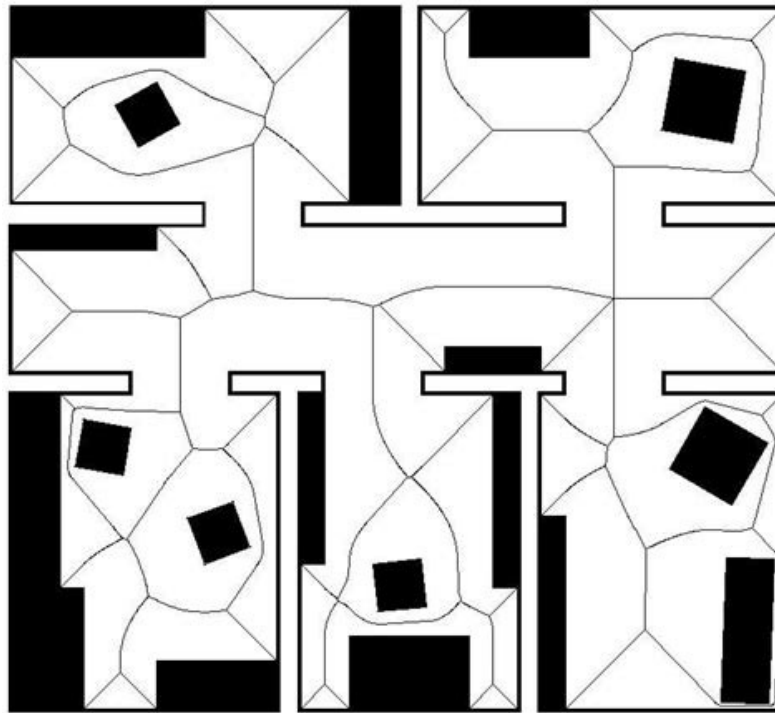


Figure 3: Voronoi roadmap.

Map & Environment Representation

Topology-based representations (Roadmaps)

- Probabilistic roadmaps (PRM)
 - Generate random configurations in C-space
 - Discard invalid nodes, i.e., $q_i \notin \mathcal{C}_{free}$
 - Connect pairs of nodes (straightlines)
 - Discard invalid edges (path with collision / no direct line of sight)

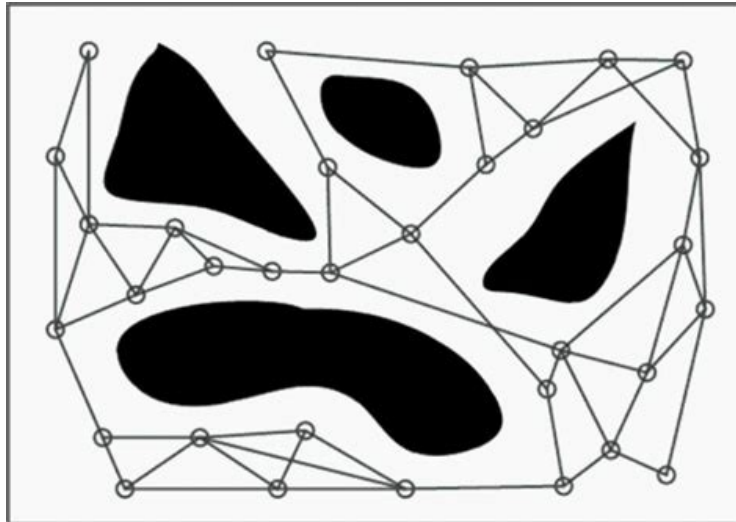


Figure 4: Probabilistic roadmap.

Map & Environment Representation

Topology-based representations (Roadmaps)

- Characteristics
 - Compact connectivity encoding
 - Good for environments with narrow passages (Voronoi)
 - Hard to generate exactly in cluttered or noisy maps
 - Requires post-processing to follow edges smoothly

Map & Environment Representation

Cell decomposition

Decompose $\mathcal{C}_{free}$ into simple, well-structured regions (cells), then construct a graph of their connectivity.

- Exact cell decomposition (polygonal maps)

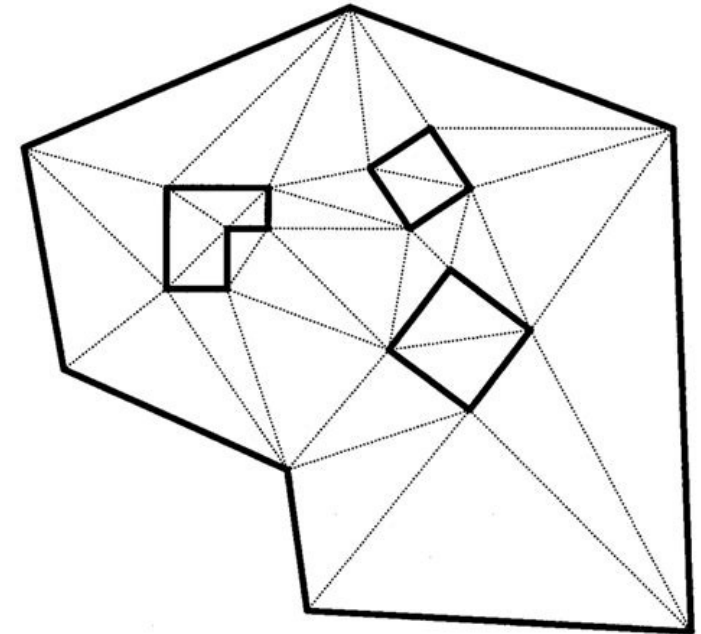
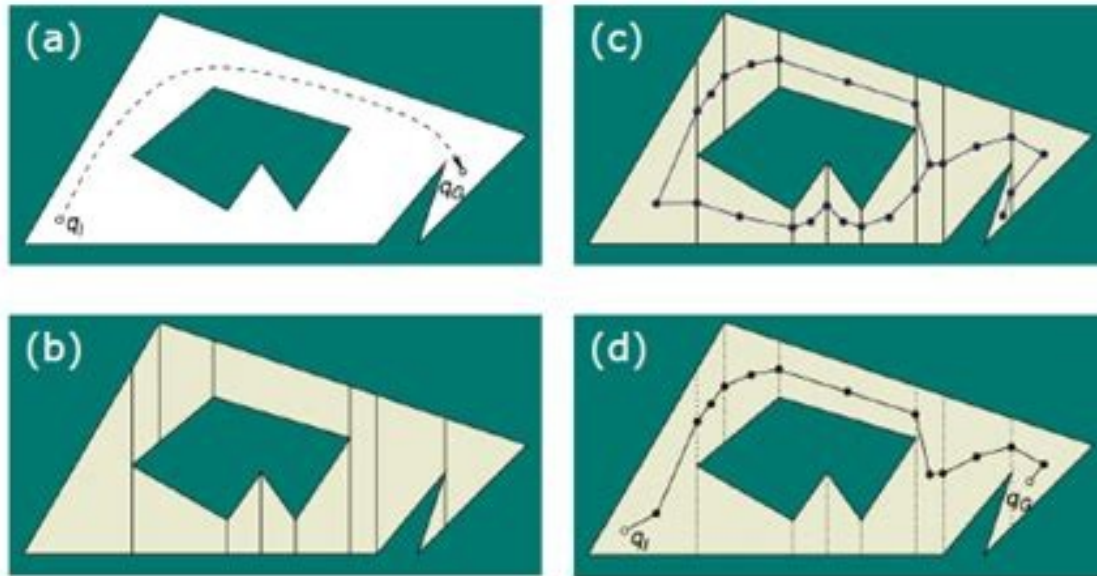


Figure 5: Trapezoidal map vs Constrained Delaunay Triangulation.

Map & Environment Representation

Cell decomposition

- Approximate cell decomposition
Recursive subdivision (quadtree, octree)

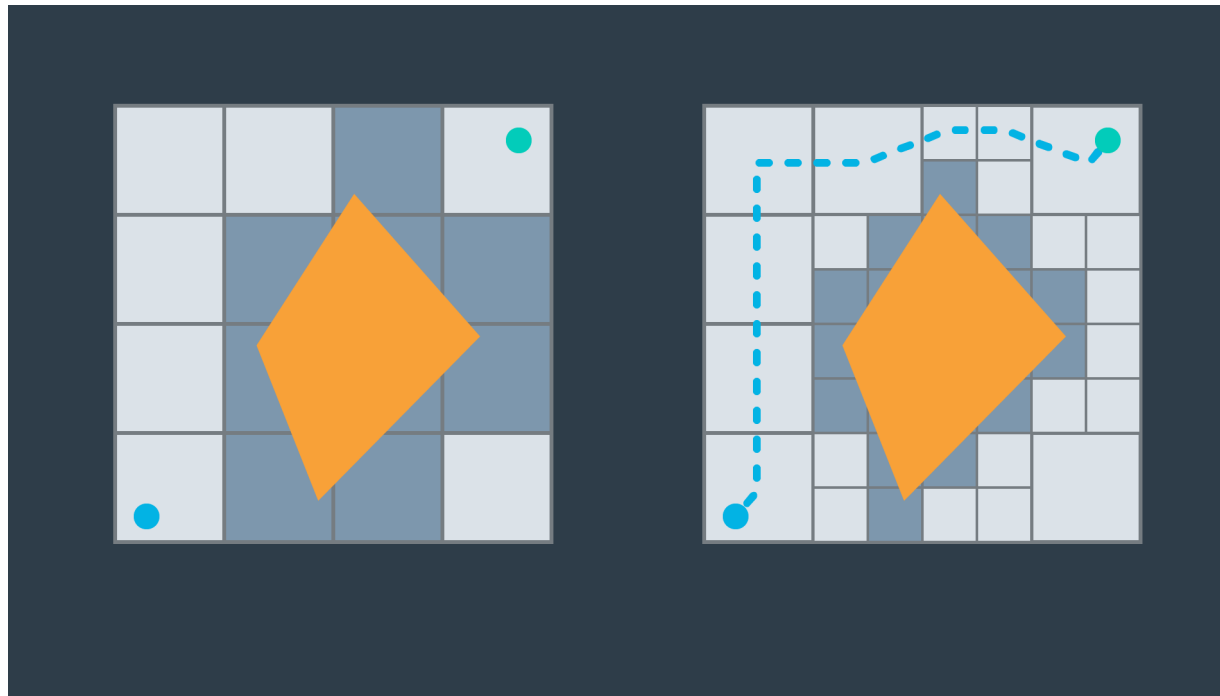


Figure 6: Approximate cell decomposition.

Map & Environment Representation

Cell decomposition

- Characteristics
 - Efficient representation for structured environments
 - Path planning reduces to graph search between adjacent cells
 - Hard to maintain exact decomposition for complex shapes
 - Path is cell-level, requires refinement
 - High DOF leads to cell count explosion

Map & Environment Representation

Visibility Graphs

Create a graph by connecting every pair of vertices (obstacle corners in C-space, start/goal configurations) that have a direct line of sight.

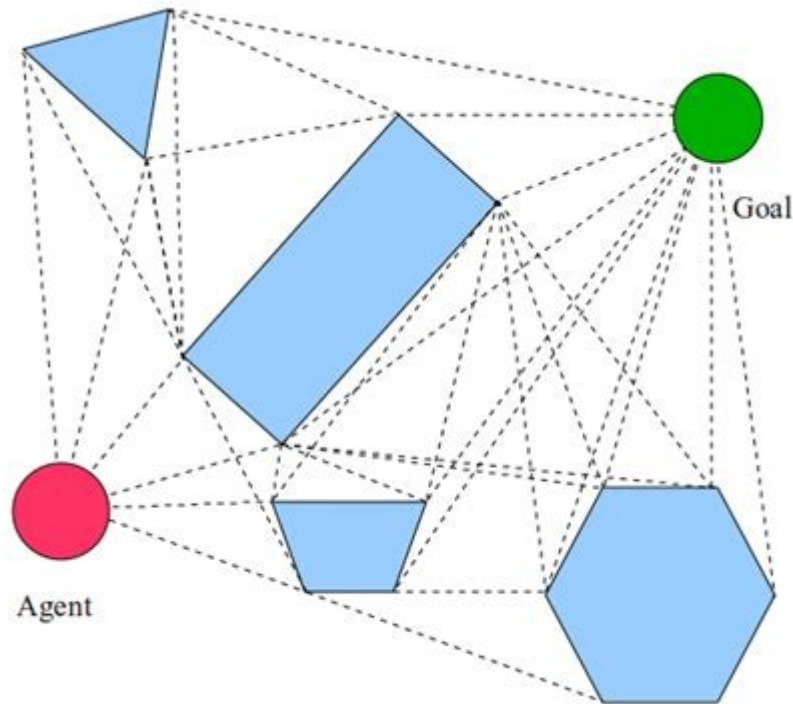


Figure 7: Visibility graph.

Map & Environment Representation

Visibility Graphs

- Produces optimal paths in polygonal environments (Euclidean metric, shortest path)
- Usually high construction cost for many obstacles
 - Minimal Construct mitigates this problem by computing only the necessary portion of the graph*
- Not collision-robust (robot hugs obstacle boundaries)
 - Obstacles must be expanded to add safety margins*
- No support for non-holonomic robots

Deterministic & Graph-based Planning

Graph search overview

- Transform the planning problem into a graph search problem
 - Nodes represent discrete configurations
Cell centers, roadmap vertices, etc.
 - Edges represents valid transitions
Neighboring cells, feasible motions, etc.
Each edge has a weight that encodes a cost (distance, energy, time, risk, etc.)
- Search a path in the graph that connects the start node to the goal node

Deterministic & Graph-based Planning

Dijkstra's Algorithm

- Computes the cost-minimal path between nodes in a weighted graph
- Foundational for deterministic, graph-based planning

```
function Dijkstra(Graph, start, goal):
    for each vertex v in Graph.Vertices:
        g[v] ← INFINITY // cost from start to v
        parent[v] ← UNDEFINED

    OPEN.add(start)
    CLOSED ← ∅
    g[start] ← 0
    parent[start] ← start

    while OPEN is not empty:
        u ← vertex in OPEN with minimum g[u]
        if u = goal:
            return reconstruct_path(parent, goal)

        OPEN.remove(u)
        CLOSED.add(u)

        for each neighbor v of u:
            if v in CLOSED:
                continue
            tentative_g ← g[u] + cost(u, v)
            if tentative_g < g[v]:
                g[v] ← tentative_g
                parent[v] ← u
            if v not in OPEN:
                OPEN.add(v)

    return FAILURE // goal not reachable
```

Deterministic & Graph-based Planning

Dijkstra's Algorithm

- Characteristics
 - Uninformed, exhaustive search (all nodes are explored evenly)
 - Optimal (requires non-negative edge costs)
 - Complete
 - Time complexity $O(|E| + |V| \log |V|)$

Deterministic & Graph-based Planning

Heuristic search motivation

- Dijkstra's algorithm explores all nodes equally which can be slow on large maps
- A heuristic can be used to estimate the cost-to-goal to guide the search
 - Reduces explored nodes dramatically
 - For optimality the heuristic must be **admissible** (never overestimate)

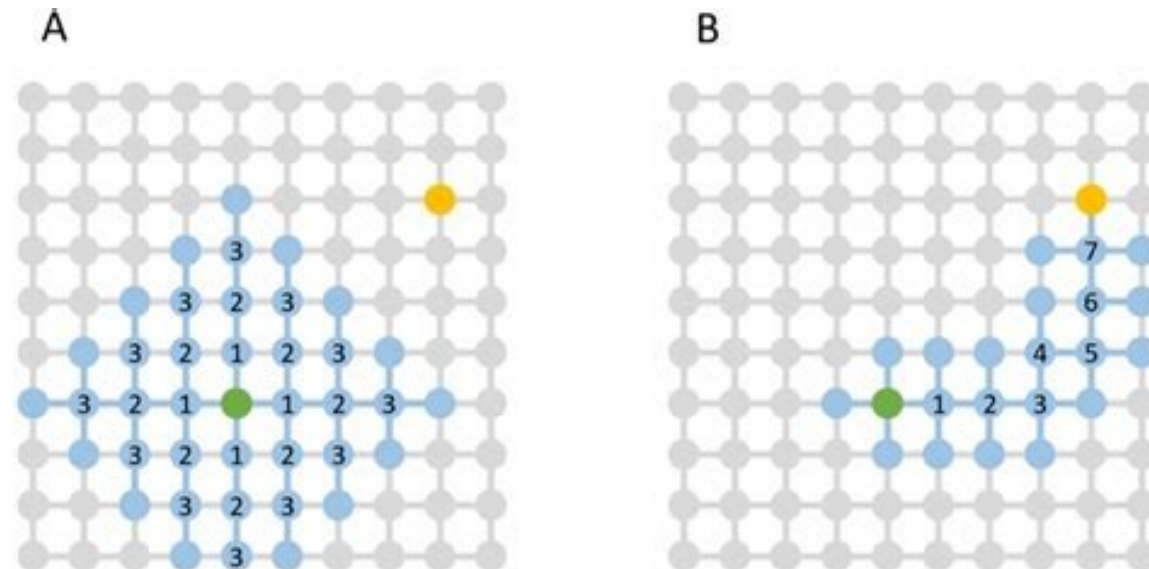


Figure 8: Dijkstra vs heuristic search.

Deterministic & Graph-based Planning

A*

- Informed search using $f(n) = g(n) + h(n)$
 - $g(n)$ is the cost from start to n
 - $h(n)$ is the cost from n to goal

```
function AStar(Graph, start, goal, h):
    for each vertex v in Graph.Vertices:
        g[v] ← INFINITY // cost from start to v
        f[v] ← INFINITY // estimated total cost (g + h)
        parent[v] ← UNDEFINED

    OPEN.add(start)
    CLOSED ← ∅
    g[start] ← 0
    f[start] ← h(start)
    parent[start] ← start

    while OPEN is not empty:
        u ← vertex in OPEN with minimum f[u]
        if u = goal:
            return reconstruct_path(parent, goal)

        OPEN.remove(u)
        CLOSED.add(u)

        for each neighbor v of u:
            if v in CLOSED:
                continue
            tentative_g ← g[u] + cost(u, v)
            if tentative_g < g[v]:
                g[v] ← tentative_g
                f[v] ← g[v] + h(v)
                parent[v] ← u
                if v not in OPEN:
                    OPEN.add(v)

    return FAILURE // goal not reachable
```

Deterministic & Graph-based Planning

A* variants

- A* is optimal and complete (under admissible and consistent heuristics)
- Several variants were proposed to improve performance by modifying the heuristic or search policy:

- Greedy Best-First search

$$f(v) = h(v)$$

Very fast but no guarantees (neither complete nor optimal)

- Weighted A*

$$f(v) = g(v) + w h(v), w > 1$$

Strongly bias the search towards the goal, much faster search, fewer expanded nodes (complete but not optimal)

Deterministic & Graph-based Planning

A* variants

- Several variants were proposed to improve performance by modifying the heuristic or search policy:

- Tie-breaking A*

When two nodes have the same f -value, prefer the one with larger g -value (deeper in the search)

Alternatively:

$$f'(v) = f(v) + \epsilon g(v)$$

Faster than regular A with the same guarantees (optimal and complete).*

- Multi-heuristic A* (MHA*)

Deterministic & Graph-based Planning

Theta*

- Theta* modifies A* by allowing line-of-sights (LOS) shortcuts between a node and its parent
- It is an *Any-angle* graph search algorithm, produces shorter and smoother paths than A*
- In theory applicable to any graph, in practice almost always implemented on grid maps
Requires a cheap LOS check between two nodes (raycasting / Bresenham)

- Principle:

Instead of only considering:

$$g(v) = g(u) + \text{cost}(u, v)$$

Theta* also considers:

$$g(v) = g(\text{parent}(u)) + \text{cost}(\text{parent}(u), v)$$

But only when $\text{parent}(u)$ and v have a direct LOS.

- Near-optimal, higher quality path, similar computational cost to A*

Fewer node expansion but more expensive LOS checks

Deterministic & Graph-based Planning

Theta*

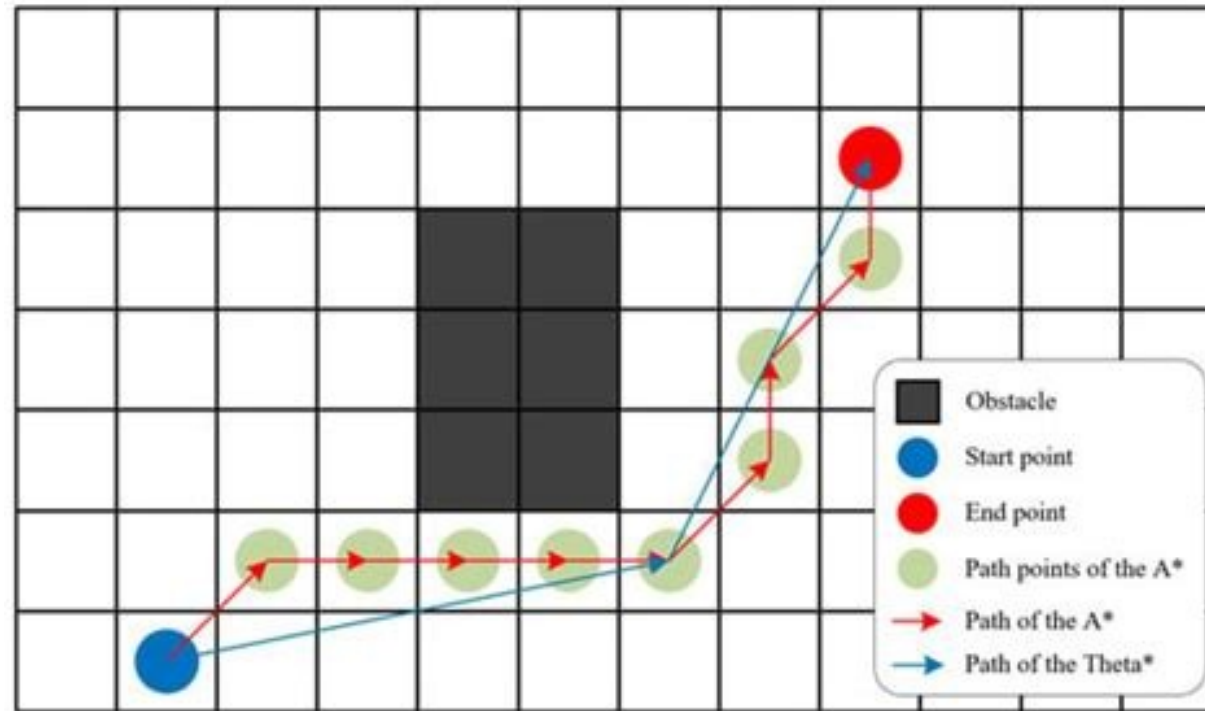


Figure 9: A* vs Theta*.

Deterministic & Graph-based Planning

Theta*

```
function ThetaStar(Graph, start, goal, h):
  for each vertex v in Graph.Vertices:
    g[v] ← INFINITY
    f[v] ← INFINITY
    parent[v] ← UNDEFINED

  OPEN.add(start)
  CLOSED ← ∅
  g[start] ← 0
  f[start] ← h(start)
  parent[start] ← start

  while OPEN is not empty:
    u ← vertex in OPEN with minimum f[u]
    if u = goal:
      return reconstruct_path(parent, goal)

    OPEN.remove(u)
    CLOSED.add(u)

    for each neighbor v of u:
      if v in CLOSED:
        continue
      if line_of_sight(parent[u], v): // Try skipping u entirely
        p ← parent[u]
        tentative_g ← g[p] + cost(p, v)
        if tentative_g < g[v]:
          g[v] ← tentative_g
          f[v] ← g[v] + h(v)
          parent[v] ← p
          if v not in OPEN:
            OPEN.add(v)
      else: // Regular A* relaxation
        tentative_g ← g[u] + cost(u, v)
        if tentative_g < g[v]:
          g[v] ← tentative_g
          f[v] ← g[v] + h(v)
          parent[v] ← u
          if v not in OPEN:
            OPEN.add(v)

  return FAILURE
```